

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ «ЛЭТИ»
ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра информационных систем

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Организация ЭВМ и систем»
Тема: Исследование видеосистемы (графический режим)

Студенты гр. 4372

Климаш И.В.

Преподаватель

Подклетнов С. Г.

Санкт-Петербург

2025

Цель работы.

Изучение работы с видеосистемой в графическом режиме, вывод графика заданной функции с масштабированием и разметкой осей.

Задание на лабораторную работу.

1. Разработать программу для вывода на экран графика заданной функции (рисунок 1).

10	$\sin^3(x) + \cos^3(x)$	$\pi/2$	6π
----	-------------------------	---------	--------

Рисунок 1 – Заданная функция (вариантное задание)

2. Произвести разметку осей и проставить истинные значения точек.
3. Найти максимальное значение функции на заданном интервале и вывести в отдельное окно на экране.

Пример запуска программы

1. При запуске программы отображается график заданной функции согласно варианту (рисунок 3):

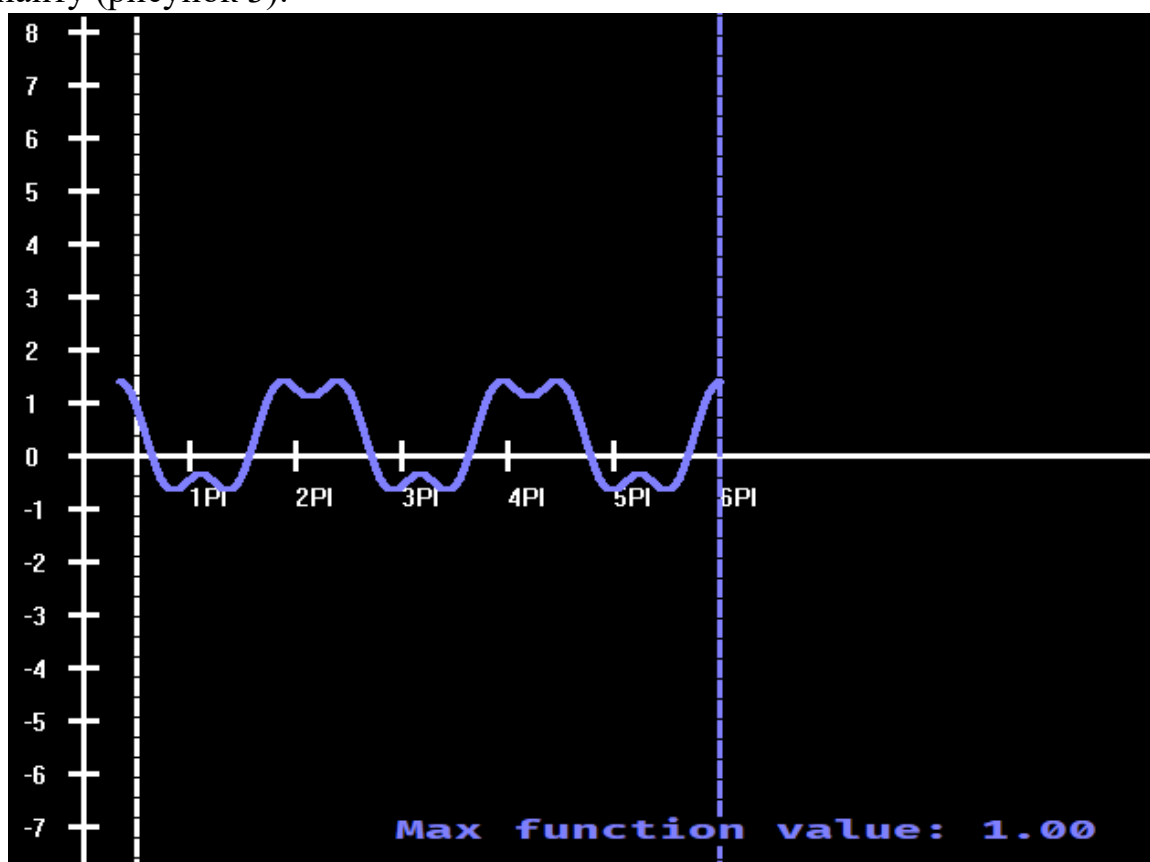


Рисунок 3 – Отображение графика функции

Краткие сведения о видеосистемах ПЭВМ и графическом режиме

Использование графики в языке C++ - это многошаговый процесс. Прежде всего необходимо определить тип видеоадаптера, затем установить подходящий режим его работы и выполнить инициализацию графической системы. После этого становятся доступными для использования функции графической библиотеки `graphics.h` для построения основных графических примитивов: отрезков прямых линий, окружностей, эллипсов, прямоугольников, секторов, дуг и т.д., появляется возможность вывода текста с использованием различных шрифтов.

Библиотека графики намного сокращает объем программирования для вывода основных графических примитивов. C++ "маскирует" многие технические детали управления оборудованием, о которых пользователь должен быть осведомлен при работе с видеоадаптером через порты или BIOS. Платой за эти удобства является значительное увеличение размера .EXE-файлов. Весь код библиотеки графики разбивается на две части: немобильную, которая зависит от типа видеоадаптера, и мобильную. Немобильная часть представляет собой так называемый .BGI-драйвер (Borland Graphics Interface). C++ включает целую коллекцию драйверов для каждого из типов адаптеров: CGA, MCGA, EGA, VGA, HERCMONO, ATT400 и другие. Совокупность внешних переменных библиотеки графики и особенностей поведения мобильных функций образует модель графики C++.

Инициализацию графической модели выполняет функция `initgraph()`. Ее вызов инициализирует графическую систему, загружая .BGI-драйвер и устанавливая видеоадаптер в графический режим. Благодаря использованию константы `DETECT` функция `initgraph()` может автоматически определить тип видеоадаптера и установить для него режим максимального разрешения. Если функции графической библиотеки больше не нужны прикладной программе, следует вызвать функцию `closegraph()` "закрытия" графического режима и возвращения к текстовому режиму. Эта функция освобождает память, распределенную под драйверы графики.

Защищенное от ошибок построение программы требует использования функции `graphresult()` после любого обращения к функциям `detectgraph()` и `initgraph()`. Эта функция возвращает значение внутреннего кода ошибки, установленного последним обращением к функциям графической библиотеки.

Рабочий код

```
#include <stdio.h>
#include <stdlib.h>
#include <graphics.h>
#include <math.h>
int main() {
    const double pi = 3.14159265358979323846;
    const double x_approach = 60 / pi;
    const double y_approach = 30;

    const int ten_num = 15;
    const float start_out = 0.5;
    const int end_out = 6;

    const int hatch_out = 58;
    const int number_out = 15;
    const int lines_out = 32; //
    double y_result;
    double x_result;
    double y_out;
    double x_out;
    double max_result = -100;
    int i;
    int graph_driver, graph_mode, graph_error_code;
    int max_x;
    int max_y;
    char symbols_out[10];
    char maximum_out[50];

    graph_driver = DETECT;
    graph_mode = VGA;
    detectgraph(&graph_driver, &graph_mode);
    initgraph(&graph_driver, &graph_mode, NULL);

    graph_error_code = graphresult();
    if (graph_error_code != grOk) {
        printf("Error: %s\n", grapherrormsg(graph_error_code));
        getch();
        return 255;
    }

    max_x = getmaxx();
    max_y = getmaxy();
    setviewport(10, 10, max_x - 10, max_y - 10, 0);
    setlinestyle(0, 0, 3);
    line(lines_out, max_y, lines_out, 0); // vert
    line(lines_out, max_y / 2, max_x, max_y / 2); // gorizont
    // Разметка по оси X
    for (i = 1; i <= end_out; i++) { // Начинаем с 1, так как 0 - это 0п
        sprintf(symbols_out, "%dPI", i);
        outtextxy(lines_out + hatch_out * i, max_y / 2 + ten_num,
symbols_out);
    }
}
```

```

        line(lines_out + hatch_out * i, max_y / 2 + ten_num / 2,
lines_out + hatch_out * i, max_y / 2 - ten_num / 2);
    }
    // Разметка по оси Y
    for (i = -number_out; i <= number_out; i++) {
        sprintf(symbols_out, "%d", i);
        outtextxy(0, max_y / 2 - hatch_out * i / 2 - ten_num / 2,
symbols_out);
        line(lines_out + ten_num / 2, max_y / 2 - hatch_out * i / 2,
lines_out - ten_num / 2, max_y / 2 - hatch_out * i / 2);
    }
    setlinestyle(3, 0, 3);
    line(lines_out + hatch_out * start_out, max_y,
        lines_out + hatch_out * start_out, 0);

    setlinestyle(0, 0, 3);
    setviewport(0, 0, max_x, max_y, 0);
    // graph
    for (x_result = start_out * pi; x_result <= end_out * pi; x_result +=
0.001) {
        y_result = pow(sin(x_result), 3) + pow(cos(x_result), 3);
        x_out = x_result * x_approach;
        y_out = y_result * y_approach;
        if (y_result > max_result) {
            max_result = y_result;
        }
        setcolor(4);
        circle(lines_out + x_out, max_y / 2 - y_out, 1);
    }

    setlinestyle(3, 0, 3);
    line(lines_out + hatch_out * end_out + 10, max_y,
        lines_out + hatch_out * end_out + 10, 0);

    sprintf(maximum_out, " ", max_result);
    setttextstyle(0, 0, 2);
    outtextxy(max_x / 3, max_y - lines_out, maximum_out);
    getch();
    closegraph();
    return 0;
}

```

Вывод

В результате выполнения работы были изучены принципы работы видеосистемы в графическом режиме и получены навыки по работе с ней.